

第1.2课

EVM存储结构

Solidity智能合约开发
从入门到精通



Storage
永久存储



Memory
临时存储



Calldata
只读数据

今天的学习内容

1



为什么要学习存储机制?

理解存储优化的重要性与经济效益

2



EVM存储架构概览

探索EVM的四种存储位置及其特性

3



Storage深入理解

永久存储的特点、成本与优化技巧

4



Memory与Calldata详解

临时存储与只读数据的特性与应用

5



三者对比与优化技巧

存储位置选择策略与Gas优化方法

6



实战与最佳实践

案例分析与Gas优化实战演练

为什么要学习EVM存储?

未优化版本

```
function getSum(uint[] memory data)
```

⚡ Gas: 25,200

VS

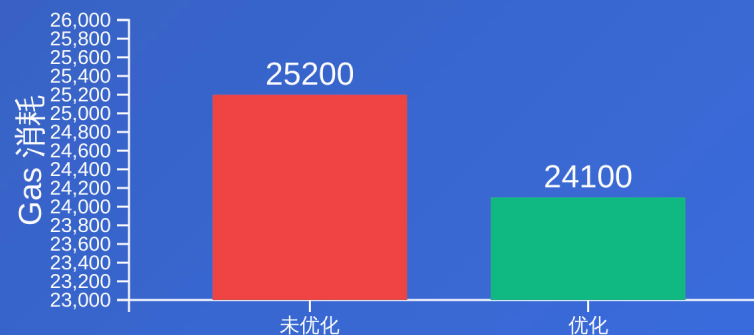
优化版本

```
function getSum(uint[] calldata data)
```

⚡ Gas: 24,100

经济效益

- ✔ 同样的功能，优化可节省 **4.4%** 的Gas成本
- ✔ 在大型项目中，优化效果累积可节省 **30%以上** 的Gas
- ✔ 实际项目中可能节省 **数百甚至数千美元** 的开发和运营费用



实际案例对比



Memory版本

```
function getSum (uint [] memory data) public pure returns (uint) {  
    uint sum = 0;  
    for ( uint i = 0; i < data.length; i++) {  
        sum += data[i];  
    }  
    return sum;  
}
```

Gas消耗: **25,200**

数组长度: 10



Calldata版本

```
function getSum(uint [] calldata data) public pure returns (uint) {  
    uint sum = 0;  
    for ( uint i = 0; i < data.length; i++) {  
        sum += data[i];  
    }  
    return sum;  
}
```

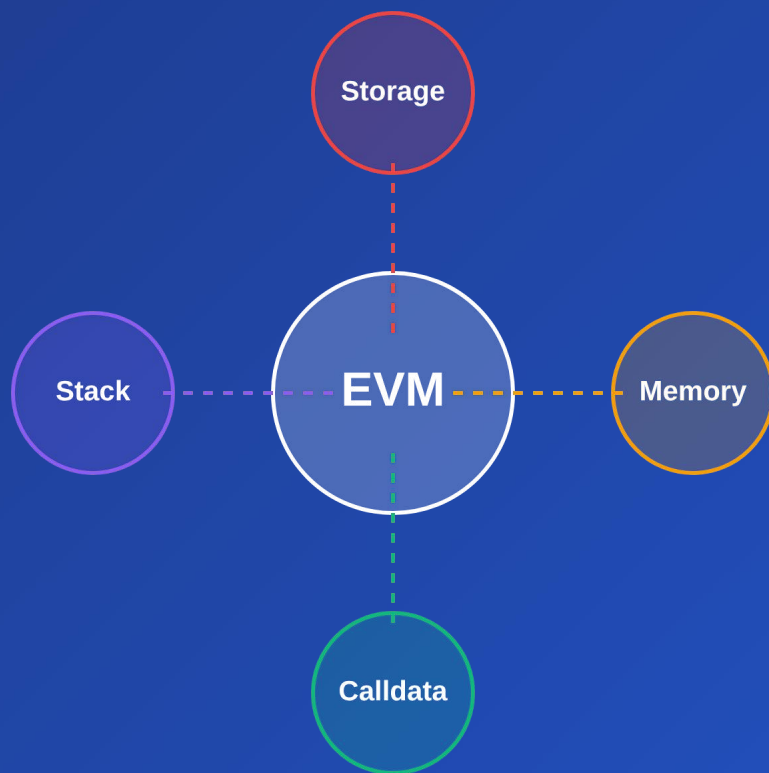
Gas消耗: **24,100**

数组长度: 10

节省: **1,100 gas | 4.4%**



EVM的四种存储位置



Storage (存储)

永久保存合约状态变量，数据存储在区块链上

⚠ 写入成本高（约20,000 Gas）



Memory (内存)

函数执行期间的临时存储空间，函数结束后清空

💡 每字节约3 Gas



Stack (栈)

执行操作和存储局部变量的区域，遵循LIFO原则

⚡ 访问速度快，成本低



Calldata (调用数据)

外部函数调用时传入的只读数据，不可修改

✅ 最经济高效的选项（约3 Gas）

类比电脑存储系统

Storage → 硬盘 (HDD)



- 永久保存在区块链
- 容量大但慢
- 最昂贵 (20,000 gas)



Memory → 内存 (RAM)



- 函数执行期间临时保存
- 速度快
- 成本中等 (3 gas)



Stack → CPU寄存器



- 超快速访问
- 容量极小 (1024个元素)
- 自动管理



Calldata → 只读光盘



- 只读, 不可修改
- 最便宜 (3 gas)
- 外部函数参数



三种存储方式对比

特性	 Storage	 Memory	 Calldata
 数据位置	区块链	内存	交易数据
 生命周期	永久	函数执行期	函数执行期
 可修改性	✓ 可修改	✓ 可修改	✗ 只读
 Gas成本	20,000 gas (写入)	3 gas/字节	3 gas/字节
 使用场景	状态变量	临时变量	外部参数

 Storage (昂贵)  Memory (中等)  Calldata (便宜)

Storage: 区块链上的永久存储

核心特性

- ✓ 永久保存: 数据永久保存在区块链上, 直到合约被销毁
- ✓ 合约级状态变量: 所有在合约中声明的状态变量默认存储在Storage中
- ⚠ 高昂成本: Storage操作是EVM中最昂贵的操作之一

使用场景

-  用户余额
-  合约状态
-  所有者地址
-  需要永久保存的数据

Gas成本分析

写入新变量 (零→非零) 约20,000 Gas

20,000

修改已有变量 (非零→不同非零) 约5,000 Gas

5,000

读取存储变量 (SLOAD)

• 冷访问 约800 Gas

800

• 热访问 约100 Gas

100

删除存储槽 (非零→零) 返还15,000 Gas

+15,000

Storage的昂贵代价

Storage操作成本

 写入新变量: 约**20,000 Gas**

 修改已有变量: 约**5,000 Gas**

 读取存储变量: 冷访问**800 Gas**, 热访问**100 Gas**

 删除存储槽: 可获得**15,000 Gas**返还

Storage操作的真正成本

仅仅存储一个数字就需要花费4美元，这凸显了Storage操作的巨大经济成本，因此在智能合约开发中必须谨慎使用并进行优化。

成本量化分析

假设:

写入一个uint256变量的Gas
消耗
20,000 Gas

Gas价格
50 gwei

ETH价格
\$4,000

成本计算:

$$20,000 \text{ Gas} \times 50 \text{ gwei/Gas} \times 10^{-9} \text{ ETH/gwei} \times \$4,000/\text{ETH} = \text{\$4 美元}$$

如何优化Storage使用



批量更新

累积结果后一次性写入，避免循环中频繁写入storage

```
for (uint i = 0; i < newValues.length; i++) {  
    values.push(newValues[i]); // 循环中多次写入  
}
```



```
for (uint i = 0; i < newValues.length; i++) {  
    tempValues.push(newValues[i]);  
}  
values = tempValues; // 一次性写入
```



使用局部变量

缓存storage变量到memory，减少storage读取次数

```
function calculate() public view returns (uint256) {  
    return myValue * 2; // 每次都读取storage  
}
```



```
function calculate() public view returns (uint256) {  
    uint256 cachedValue = myValue; // 缓存到局部变量  
    return cachedValue * 2;  
}
```



变量打包

合理安排小类型变量，减少storage slots使用

```
struct BadStruct {  
    uint256 largeVar; // 槽0  
    uint8 smallVar; // 槽1，浪费31字节  
}  
  
// 浪费storage slot
```

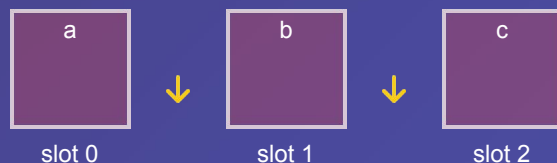


```
uint256) {  
    // 局部变量  
  
    struct GoodStruct {  
        uint8 varA; // 槽0  
        uint8 varB; // 槽0  
        uint256 largeVar; // 槽1  
    }  
  
    // 优化变量顺序，减少slot使用
```

变量打包节省Gas

✗ 不好 - 占用3个slot

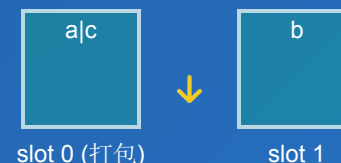
```
contract Bad {  
    uint8 a;    // slot 0  
    uint256 b;  // slot 1  
    uint8 c;    // slot 2  
}
```



⚠ 浪费存储空间, 增加Gas消耗

✓ 好 - 只占用2个slot

```
contract Good {  
    uint8 a;    // slot 0  
    uint8 c;    // slot 0 (打包)  
    uint256 b;  // slot 1  
}
```



★ 节省1个storage slot, 约20,000 Gas

变量打包规则:

- ✓ 变量必须连续声明, 中间不能有大类型变量间隔
- ✓ 只有静态大小类型可以打包 (排除mapping和动态数组)
- ✓ 当累计大小超过32字节时, 使用新的存储槽

Memory: 函数内的临时存储

特点

-  **临时性:** 仅在函数执行期间存在，函数调用结束后自动清空
-  **可修改性:** Memory中的数据可以被修改
-  **引用类型需显式声明:** 数组、结构体、字符串等需要显式使用memory关键字
-  **成本较低:** 每字节大约消耗3 Gas，远低于Storage

Gas成本对比

Memory:  ~3 gas/字节

Storage:  ~20,000 gas/
写入

使用场景

函数参数

用于传递引用类型的函数参数，如
`string memory _input`

临时变量

存储函数执行过程中的中间结果或临时数据

复杂计算

处理大型数据结构时的中间结果存储

不需永久保存

任何不需要在交易或函数调用之间持久化的数据

代码示例

```
function processString(
    string memory _input
) public pure returns (string memory) {
    string memory result =
        string.concat("Hello, ", _input);
    return result;
}
```

Memory使用示例

</> 函数参数与Memory

```
function processString (  
    string memory _input  
) public pure returns (string memory) {  
    // _input在memory中  
    string memory result =  
        string.concat( "Hello, " , _input);  
    return result;  
    // 函数结束, _input和result消失  
}
```

// Gas消耗: 约 23,000 gas



使用**memory**关键字声明函数参数, 表示该参数存储在Memory中



Memory操作每字节消耗约3 Gas, 远低于Storage

🏗️ Memory特性与生命周期



临时性

数据仅在函数执行期间存在, 函数调用结束后自动清空



可修改性

Memory中的数据可以被修改, 适用于复杂计算



引用类型需显式声明

对于数组、结构体、字符串等引用类型, 需显式使用memory关键字



成本较低

每字节大约消耗3 Gas, 远低于Storage的开销

Calldata: 最省Gas的选择

特点

- ✓ 只读，不可修改
- ✓ 最便宜的数据存储方式
- ✓ 外部函数参数专用
- ✓ 函数执行结束后自动消失

Gas成本

每字节成本:

3 Gas



0

5,000 Gas

 是EVM中最经济高效的数据存储方式

何时使用

外部函数的数组/字符串参数

优先使用calldata避免不必要的内存复制

不需要修改参数

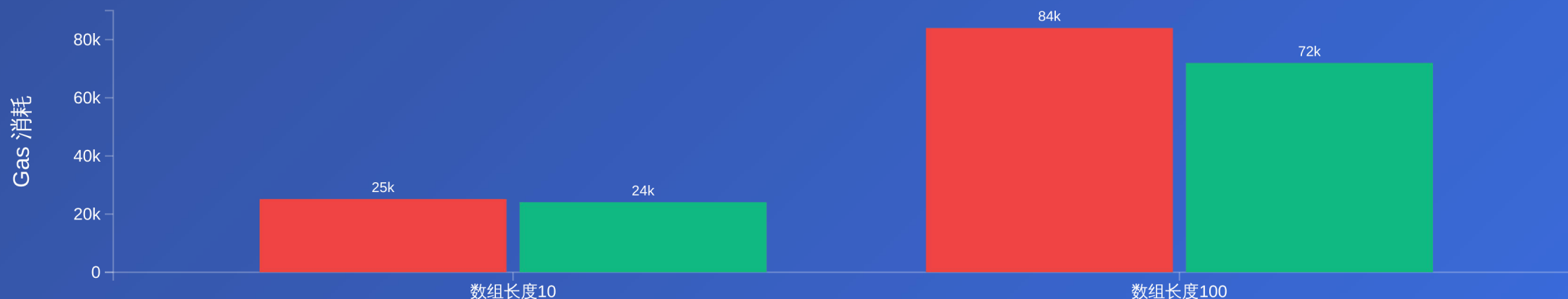
如果函数参数仅用于读取，不涉及修改操作

追求极致Gas优化

在对Gas成本有严格要求的场景下首选

Memory vs Calldata 性能对比

测试：数组求和



Memory Calldata

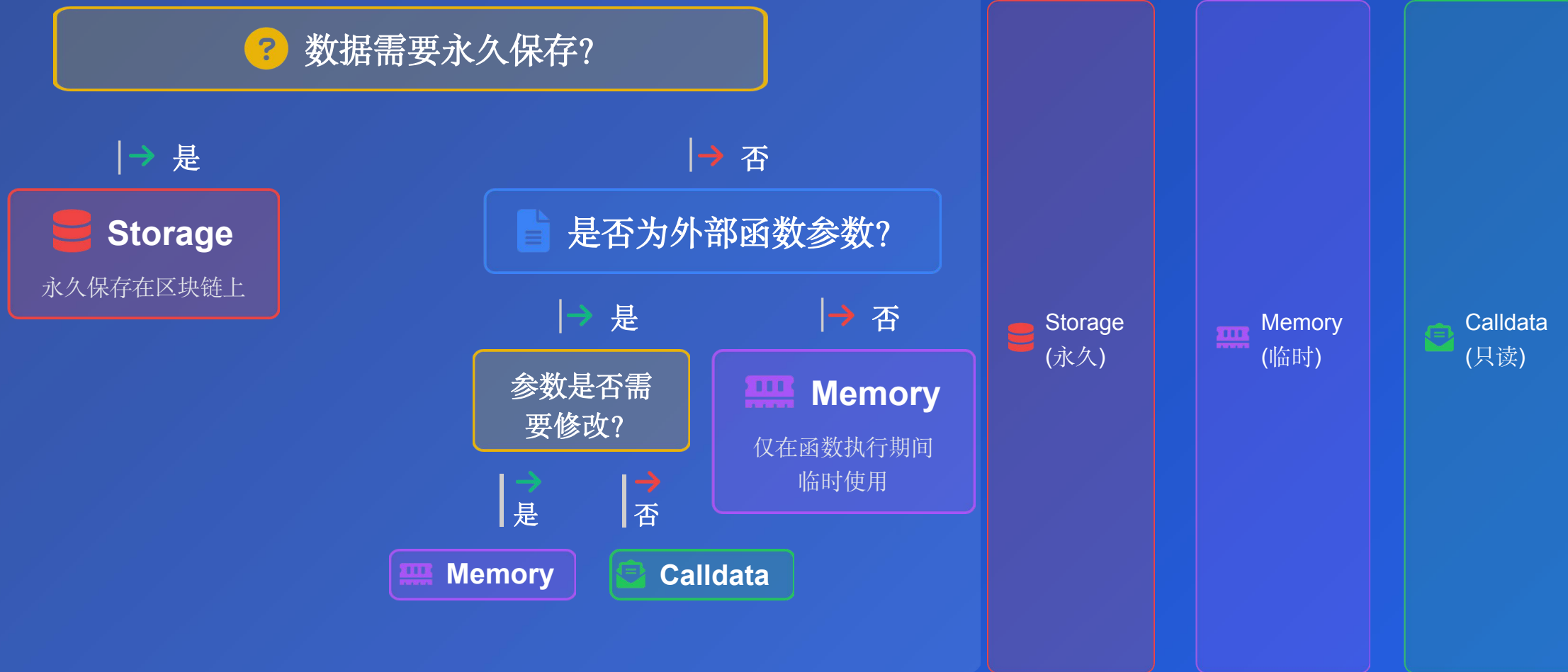
→ 数组长度10

- Memory: 25,200 gas
- Calldata: 24,100 gas
- 节省: 1,100 gas (4.4%)

→ 数组长度100

- Memory: 84,000 gas
- Calldata: 72,000 gas
- 节省: 12,000 gas (14%)

如何选择存储位置？



实战：优化前后对比

✖ 未优化合约

```
function processArray( uint[] memory data) external {  
    for ( uint i = 0; i < data.length; i++) {  
        array.push(data[i]); // 循环中直接写storage  
    }  
}  
  
function getLength() external view {  
    for ( uint i = 0; i < array.length; i++) { // 每次读取length  
        // 处理逻辑  
    }  
}
```

Gas消耗: **180,000**



✔ 优化后合约

```
function processArray(uint[] calldata data) external {  
    uint len = array.length; // 缓存length  
    for ( uint i = 0; i < data.length; i++) {  
        array.push(data[i]);  
    }  
}  
  
function getLength() external view {  
    uint len = array.length; // 缓存storage变量  
    for ( uint i = 0; i < len; i++) {  
        // 处理逻辑  
    }  
}
```

Gas消耗: **165,000**

优化技术:

✔ 使用calldata参数

✔ 缓存storage变量

✔ 缓存length

Gas节省: **15,000 gas (8.3%)**

节省15,000 ga

Gas优化最佳实践



外部参数用calldata

只读引用交易数据，避免内存复制

```
function process(uint[] calldata data) external pure
```



缓存storage变量

减少昂贵的SLOAD操作

```
uint256 cached = myValue; // 读取一次 return cached  
* 2; // 后续使用缓存
```



批量操作

避免循环中频繁写入storage

```
// 累积后一次性写入 values.push(newValues[i]);
```



变量打包

多个小变量共用一个storage slot

```
uint8 a; uint8 c; uint256 b; // 打包到slot0
```



使用constant和immutable

不占用storage空间，访问成本低

```
uint256 constant MAX = 1000; address immutable  
owner;
```



避免外部调用

外部函数调用比内部调用消耗更多Gas

今日作业

1 优化合约

给定合约存在存储效率问题，要求优化至少20%的gas

💡 提示:

使用calldata替代memory

缓存storage读取

减少storage写入次数

2 思考题

- ❓ 为什么calldata比memory便宜?
- ❓ 什么情况必须用storage?
- ❓ 如何缓存storage变量以优化?

下节课预告

📅 第2.1课：函数基本语法

我们将学习：



Solidity数据类型详解

值类型 vs 引用类型



运算符的使用

算术、比较、逻辑运算



类型转换技巧

安全的类型转换方法



本课重点回顾

- Storage、Memory、Calldata的特性与成本差异
- Gas优化技巧与最佳实践
- 变量打包与缓存策略
- 如何选择合适的存储位置



互动时间

有什么问题？现在是提问的好时机！



点赞



收藏



转发

感谢观看！理解存储="写出高质量合约"